Formalizing Complexity Theory

Christian Reitwiessner

leaning in 2026, Berlin

2026-03-12

Complexity Theory

Complexity theory studies the relation between various *resource bounds* for computation:

Time	number of computation steps
Space	number of memory cells used
Nondeterminism	verifying a proposed solution
Randomness	
Quantum	

Not so much about finding efficient algorithms for given problems, but rather if e.g. randomness or nondeterminism acutally provide an advantage and what the trade-offs between resource bounds are.

The P vs. NP Question

$\mathbb{P}$ Yes-no-problems solvable in polynomial time.

$\mathbb{NP}$ Problems whose solutions can be *verified* in polynomial time.
$L \in \mathbb{NP} \iff \exists W \in \mathbb{P}, (x \in L \iff \exists^p y, (x, y) \in W)$

$$\mathbb{P} \stackrel{?}{=} \mathbb{NP}$$

- One of the seven Millennium Prize Problems.
- If $\mathbb{P} = \mathbb{NP}$: cryptography, optimisation, formalizing mathematics would be revolutionised.
- Widely believed: $\mathbb{P} \neq \mathbb{NP}$, but no proof in sight.

Decades of Stagnation

- $\mathbb{P} \stackrel{?}{=} \mathbb{NP}$ only one example
- Most existing techniques cannot solve the hard problems in complexity ("natural proofs", "relativization", "algebrization")
- Solution has to use mathematical insights far removed from actual machines.
- Community largely gave up on a near-term resolution.

Space-efficient computations

- Obvious: $\mathsf{DTIME}(T) \subseteq \mathsf{DSPACE}(T)$
- Hopcroft–Paul–Valiant (1975): $\subseteq \mathsf{DSPACE}(T/\log T)$
- **Ryan Williams (2025):** $\subseteq \mathsf{DSPACE}\left(\sqrt{T} \cdot \mathrm{polylog}(T)\right)$
- Key ingredient: encode many values in the same memory cells using polynomials over finite fields and erase anything that is possible to re-compute in the future (Cook-Mertz 2023).

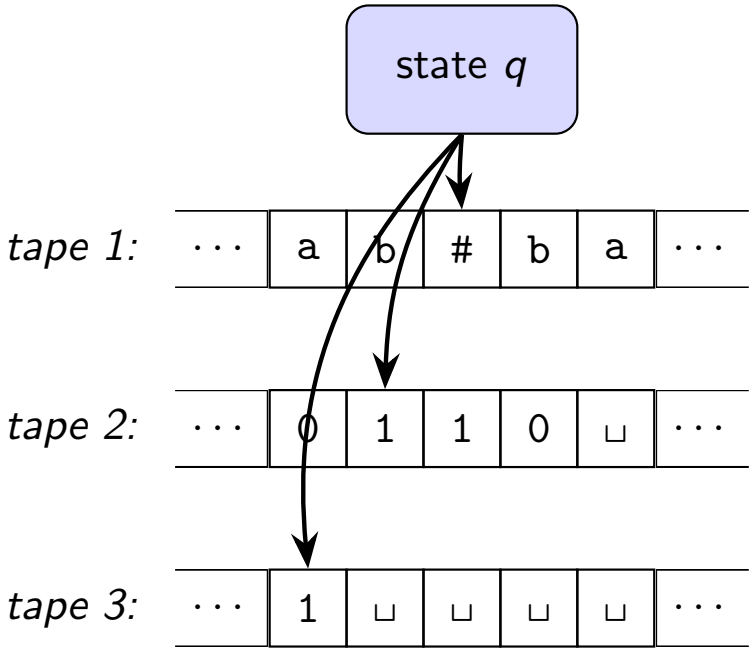
Personal Perspective

- I learnt about Williams' result and Lean roughly at the same time.
- Let's formalize the result before someone else does!
- I had no idea.
- First obstacle: cannot even *state* the theorem.
- Complexity theory is not formalized at all in Mathlib / Cslib
- Before Gäher-Kunze-2021, complexity theory was not formalized (down to the machine model) in any proof assistant.

On to work!

Turing Machine

- For computability, the computational model is largely irrelevant.
- For complexity, it is **not**—especially for space complexity.
- Williams' result is only known for Turing machines (local computation), and not for RAMs or other models.
- It gets tricky below linear space, but logarithmic space is one of the most important classes.



State of the Art

- Generally seen as very difficult to formalize, several discussions in Zulip.
- Gäher and Kunze (2021) formalize computational complexity using Turing machines in Coq for the first time, down to the machine model.
- Their approach to use weak call-by-value λ-calculus does not work for logarithmic space.
- Mathlib has Turing machines, but only single-tape and with focus on computability.

Challenges: Why Is It So Hard?

- Many hand-wavy arguments in standard textbooks.
- Turing machines specified in prose.
- Frequent informal estimations that are hard to make rigorous.

CANYIELD = "On input c_1, c_2, and t:

1. If $t = 1$, then test directly whether $c_1 = c_2$ or whether c_1 yields c_2 in one step according to the rules of N. *Accept* if either test succeeds; *reject* if both fail.
2. If $t > 1$, then for each configuration c_m of N using space $f(n)$:
3. Run CANYIELD$(c_1, c_m, \frac{t}{2})$.
4. Run CANYIELD$(c_m, c_2, \frac{t}{2})$.
5. If steps 3 and 4 both accept, then *accept*.
6. If haven't yet accepted, *reject*."

Current Progress

Formalized in `https://github.com/crei/cslib` on top of Cslib.

- Exploratory approach:
  - `sorry` proofs for the "obvious" but technically difficult parts
  - try to have good `simp` and `grind` proofs for composition results
- Elementary Turing machines and combinator tools.
- Currently trying to prove Savitch's Theorem, $\mathrm{NSPACE}(S) \subseteq \mathrm{DSPACE}(S^2)$, i.e. a space-efficient graph reachability algorithm)

TM DSL

The DSL evaluates to regular Turing machines but operates on lists of words instead of tapes.
Similar to WHILE / LOOP programs, but with stacks instead of registers.

Example: isZero machine

```
def isZero (i : Fin k) : MultiTapeTM k Symbol :=
  ite i (pop i ; push i []) (pop i ; push i [1])

theorem isZero_eval_list {i : Fin k}
  {tapes : Fin k -> List (List Symbol)} :
  (isZero i).eval_list tapes = .some (
    Function.update tapes i (
     (if (tapes i).headD [] = [] then
        [1]
      else
        []) :: (tapes i).tail)) := by
  simp [isZero]; grind
```

Next Tasks

- For sub-linear space, we probably cannot stay with stacks but need to use arrays.
- Loops are not automatic but OK, is there a better way?
- Can we map recursive algorithms to Turing machines?
- Find a good trade-off between precision and usability for space estimation.